

Evaluation of Serial Communication Protocols for Integrated Circuits

Mikael Larsson

Luleå University of Technology

MSc Programmes in Engineering

Department of Computer Science and Electrical Engineering
Division of EISLAB

Abstract

The use of predefined standards of communication should be a great aid to the designer of modern digital systems. However, the sheer amount of standards available today may have the opposite effect. This thesis aims to give an overview of the current situation for digital serial communication methods for integrated circuits.

Two methods of communication, I²C and SPI, are dealt with in more depth. Practical usage are demonstrated when circuits using these are connected to a FPGA device.

Compiled by/Dept Mikael Larsson / RTEDS	Checked by/Dept Krob / RTEDS	Date 2005-04-18	Edition 2	Document No TR-20050019
Approved by/Dept LII / RTEDS		Document Status Released		Classification Unclassified

Evaluation of serial communication protocols for integrated circuits

Mikael Larsson



Luleå University of Technology
Dept: EISLAB
Examinator: Per Lindgren

Date	Edition	Document No
2005-04-18	2	TR-20050019
Document Status	Classification	
Released	Unclassified	

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

TABLE OF CONTENTS		Page No
1	Scope.....	5
1.1	Identification	5
1.2	System overview	5
1.3	Document overview	5
2	Referenced documents	6
3	System-wide design decisions	8
3.1	Serial principals.....	8
3.2	Case studies	11
3.3	Choice of components and serial standard.....	15
3.4	Functional description of lab implementation.....	17
4	System architectural design.	23
4.1	System overview	23
4.2	System components.....	23
4.3	Concept of execution.....	29
4.4	Interface design	35
4.5	Programmable logic requirements	36
5	Requirements traceability	38
6	Notes	39
6.1	Explanation of terms:	39
6.2	Problems encountered and how to avoid them	40
Appendix A	Error codes	42
Appendix B	P160 board layout	43
Appendix C	External circuits connection.....	44

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

1 Scope

1.1 Identification

This document was produced as a masters thesis on the subject of serial communications between digital components. Its goal is to provide an overview of the market of serial components and a description of different interfaces and thereby give some assistance to system designers. Some components and interfaces considered particularly interesting will also be demonstrated by simulation and implementation.

1.2 System overview

Due to the large amount of standards for communication that are currently in use, this thesis has been limited to serial communication between digital components without the aid of stand alone drivers. Within this scope, two major families are identified and examined in greater detail and implemented in VHDL. Other standards are described shortly and with the aim of covering those that may be of interest.

1.3 Document overview

Chapter 3 gives a functional description of a selection of serial standards. It also describes the lab part of the thesis from a functional perspective and attempts to give sufficient instructions on how the construction is used to enable the reader to repeat the experiments.

Chapter 4 describes the structures within the lab part with details on some of the VHDL building blocks as well as an structural oversight of the C-code for the software.

Chapter 5 states the requirements traceability.

Chapter 6 contains notes and explanations that aims to increase the reading value for some readers. The information here is not to be seen as a part of the actual report and can therefore not be assumed traceable through referenced documents.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

2 Referenced documents

Standard specifications:

In table **Table I: Document names and sources** the standard specifications in those cases they could be found. In other cases application notes with sufficient documentation on the protocol have been used for reference.

Table I: Document names and sources

Protocol	Document	Comment
I ² C	I ² C bus specification	www.semiconductors.philips.com
I ² S	I ² S bus specification	www.semiconductors.philips.com
SMBus	System management specification v1.1	www.smbus.org
LVDS	TIA/EIA-644-A	www.tiaonline.com (at a cost)
Microwire	Application Note 452 (AN-452)	www.national.com
Microwire/PLUS	Application Note 479 (AN-479)	www.national.com
RS232	EIA/TIA-232	www.tiaonline.com (at a cost)
SPI	Using the Serial Peripheral Interface to communicate between multiple micro-computers (AN-991)	www.freescale.com (formerly www.semiconductors.motorola.com)
McBSP	Multichannel Buffered Serial Port Reference Guide	Available from texas at www.ti.com Literature number: SPRU592D
CAN	CAN specification	www.can.bosch.com
LIN	LIN specification package	www.lin-subbus.org
1553B	MIL-STD-1553B	www.defencelink.mil

Other:

Digital system design with VHDL - Mark Zwolinski - ISBN 0-201-36063-2

Date	Edition	Document No
2005-04-18	2	TR-20050019
Document Status	Classification	
Released	Unclassified	

Virtex-II V2MB1000 Development Board User's Guide v1.3 - Memec - PN# DS-MANUAL-V2MB1000

P160 Prototype Module User's Guide v1.0 - Memec - PN# DS-MANUAL-MBEXP2

Virtex-II Platform FPGAs complete data sheet - Available from Xilinx corporation (www.xilinx.com)

QExtSerialPort - Serial port library for Qt (www.sourceforge.net/projects/qextserialport)

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

3 System-wide design decisions

3.1 Serial principals

3.1.1 Serial vs parallel communication

When designing a standard for data transfer, one of the most obvious design choices is between transmitting the data in parallel or serial. Not to long ago, the parallel method was the most popular way of transferring large amounts of data. This has however gradually changed over the last decade and almost all new communication protocols features some form of serial transmission of data. This can be considered as a natural result of the evolution in the computer industry, where more and more transistors can be fitted on the same chip, running at ever greater speeds at relatively low costs.

One reason for this is speed. The problems with the use of parallel transmission is increasing with the rise in clock frequency as phase shifting and overhearing becomes more troublesome.

These problems can be reduced with serial communication, as fewer wires means that less resources has to be given to adjusting the length of wires and compensating for timing differences in electronics. Fewer wires also means fewer sources of electromagnetic fields, and the remaining ones can often be better insulated at a more reasonable cost.

Another reason is the cost of pins and cables. Since the cost per pin on an IC circuit can be quite expensive, it is often more cost effective to use serial communication with fewer pins, even though this may increase the demands for speed and complexity of the logic handling the interface. It also reduces the cost and complexity of circuit board design as well as the cost of cables, connectors and assembly.

Ironically, as demands for even greater speeds come to rise, the industry has begun to once again looking at the possibility of sending data in parallel, trying to get the best from both worlds.

This is being done by using multiple serial communication lines in parallel, sending information that is time wise independent of each other thous avoiding the timing problem with phase skew seen in parallel buses. One example of this is PCI Express (not to be confused with PCI-X), where a different number of serial channels can be used in parallel depending on bandwidth demands.

The problem with electromagnetic fields generated and absorbed by wires running in parallel can partially be overcome by differential signalling. This means that two wires are used for one signal, one carrying the inverse waveform of the other. Incoming fields will affect the signal in both wires the same amount and the difference between the two, which holds the information transmitted, will therefore remain largely unaffected.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Long cables for this kind of signal often consists of twisted pairs (TP cable) of wires and the generated field from each wire such a pair is thereby to a high extent cancelled out by the other one.

This method is used by LVDS and Ethernet which is described briefly in **Choice of communication standard for a design** on page 15, and also in the new SATA interface for storage devices in personal computers.

3.1.2 Bused serial architectures

Whenever a design requires a protocol for communication, design decisions regarding this protocol will affect the complexity of the design. One of the greatest factors in complexity is whether the protocol should support a true bus architecture, that is, should it be possible to connect two or more slave units (assuming a master-slave approach) to the same wires and still communicate with them independently of each other.

As will be further discussed later, this one of the major differences between the two special case studies made in this thesis, the I²C and the SPI families of protocols.

The reason for the increase in complexity is that the true bus protocols normally use some form of addressing scheme. This is done either by sending the addresses on separate wires, or, sending the addresses on the same wires as the data. And given the reasons for using serial communication in the first place, it is not hard to see why the later option have become very popular.

It is however inevitable, that mixing addresses and data on the same wires will require somewhat complex designs of the logic handling this. To further complicate the issue, addresses and data are sometimes mixed up with even more information, such as direction of transfer, or bus speed to be used.

Communication in this manner is often hard to realize with normal logic constructions and its sequential nature often leads to the use of microcontrollers/processors to enable a software implementation of the protocol.

There are ways to reduce the amount of different information on the wires, but this is often accomplished by either adding more wires and/or making the protocol less flexible.

3.1.3 Overview of common standards

The following table gives a brief overlook on some of the most common standards used for serial communication between components today. The intent is not to provide a complete list of serial interfaces, since just adding those found in a modern PC would nearly to double the list (USB, firewire, SATA, RS232 and so on), but

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

rather give a oversight on those that may be most interesting when creating a design.

Table II: Overview of serial standards

Name	Creator	Description
I ² C	Philips	Inter IC Communication. A 2-wire true bus standard currently (v2.1) specifying 3 speed modes (Standard - 100kbit/s, Fast - 400kbit/s and High speed - 3,4mbit/s).
SMBus	Intel	System Management Bus. Originally created for use in Smart Battery System (SBS), SMBus conforms to standard speed I ² C with minor differences.
CBus	Philips? (not confirmed)	3-wire standard which still is mentioned in the I ² C standard as a compatible format. Uses a third wire "DLEN" as a "slave select" (see 3.2.1 The SPI group). Used in philips LCD-driver family PCF21xxC.
SensorPath	National Semiconductors	1-wire system mainly developed to replace/relieve SMBus on motherboards for temperature surveillance. Components that support this protocol is therefore mainly thermometers, fan control units and interface drivers.
SPI	Motorola	Serial Peripherals Interface. One way 3-wire or two way 4-wire standard. Does not feature true bus architecture but several slaves can be used with multiple slave select wires. Very simple, easy to implement and popular protocol.
3-Wire	Maxim	Maxim 3-wire works in a way very similar to SPI with the big difference being only one data wire used for data in both directions. Even so, 3-wire can be considered as partially compatible with the SPI-family since there are ways to bypass this difference ¹⁾ .
I ² S	Philips	Inter IC Sound bus. 3-wire standard created for audio applications. Even though the name reminds of I ² C, the two standards have very little in common. I ² S operates in a manner much closer to SPI and is often supported in parallel with SPI by AD/DA converters that are suitable for audio.
Microwire	National Semiconductors	Conforms to what is known as SPI mode 0 (trig on positive flank, no clock phase shift).
McBSP	Texas Instruments	Multi Channel Buffered Serial Port. 7 pin Interface standard found on many TI DSPs. While not a serial communication standard as such, the interface does specify the possibility to use 4 of these pins to runs SPI in either master or slave mode.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Table II: Overview of serial standards

Name	Creator	Description
LVDS	Fairchild	Low Voltage Differential Signalling. A highspeed point to point communication standard typically used for backplane communication. Specifies speed up to 644mbps, although even higher transfer rates seem to be commonly used. M-LVDS (Multi point-LVDS) is an up coming extension for bused use.
CAN	Bosch	Controller Area Network (ISO11898). Advanced data bus for the automotive market mainly used in europe. Uses two wires and operates in speeds up to 1mbit/s and supports realtime tasks with the TTCAN (Time Triggered CAN) extension.
LIN	The LIN consortium. (Audi, VW, Volvo, Daimler-Chrysler and BMW)	Local Interconnect Network. A single wire system that aims at being a low cost competitor to CAN. Operates at up to 20kbit/s. The LIN bus is normally used for simpler tasks in cars, such as light control, environment sensors and door locks. Sometimes used in parallel with CAN.

1) See "Maxim application note 85: Interfacing the DS1620 to the Motorola SPI Bus"

3.2 Case studies

From this vast number of standards for serial communication on the market today, two groups have been chosen for more careful studying in this thesis.

When looking at components that does not need an external driver to connect to the serial interface, the options are reduced. There are a number of different standards, but many are either too focused on a certain area of interest (for instance Maxim 1-wire, used widely in weather stations), or does not provide a sufficiently large selection of components to cover the needs of many designs (for instance Maxim 3-wire).

For a standard to become popular, it needs to be available on a large selection of components and preferably be supported by a number of large actors on the market, or be compatible with one or more standards that already fulfil these criteria. This has lead to two major groups of standards on the market:

- The SPI group (SPI/Microwire/McBSP)
- The I²C group (I²C/SMBus/CBus)

Within each family, different standards and version thereof exists. Never the less, the possibility to mix components within a family is the reason for this grouping.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

This also indicates that although each standard has its own pros and cons, the big differences, and thereby much of the design choice is, between the families.

3.2.1 The SPI group

The members of what in this thesis are referred to as the "SPI group" or "SPI family" are SPI, QSPI, Microwire, MicrowirePlus, McBSP and to a certain extent, Maxim 3-wire¹⁾.

The components in the SPI groups normally use 4 wires for two-way communication (or 3 for one-way) but some components can use more wires for various signals. The function of the four basic wires however is normally not affected by such additions. All wires are one way only unlike the wires in I²C group (Maxim 3-wire is an exception to this). The naming varies between the different standards, but the basic function is the same:

SCLK - Shift clock output from master to all slaves, indicating when a bit is to be transferred. This applies to both output and input if they are used in the design.

MISO - Master In Slave Out, data input to the master from one or more slaves. If a slave is not enabled through the SS wire, it needs to set this output in a high impedance state since more slaves maybe connected.

MOSI - Master Out Slave In, data output from the master connected to the input on every slave. The data sent on this wire will be received by the slave currently selected.

SS - Slave Select wire that enables a slave for input/output. There is a separate SS wire from the master to each slave device, limiting the number of possible slaves to the number of available output pins on the master. It also causes the wiring to grow with the addition of every slave device.

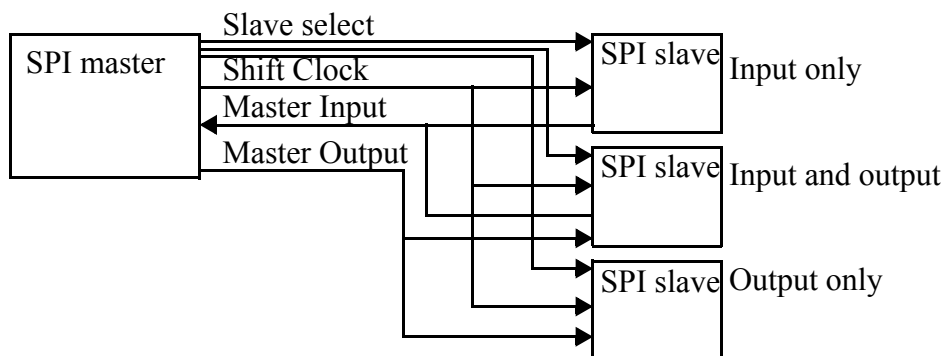
Components in this group are sometimes referred to as "DSP-compatible", indicating that many DSPs supports some form of this interface. McBSP is for instance the interface used in many DSPs from Texas instruments, it implements both the master and the slave part of SPI as well as other protocols such as AC97.

Among the SPI groups advantages is the vast number of components available and also that the protocol is simpler than that of I²C group, with less overhead as a result. The number of wires used in SPI depends on how it is being used.

To be noted is that the standards in this groups does not offer a true bus architecture. A separate wire from the master to each slave component is used to decide which slave is currently addressed. This means that the number of pins and wires used for communicating with many slave devices are much higher than in a similar setup within the I²C group, but it also means that no bandwidth is used for sending addresses, thus reducing the overhead. In combination with the separate wires for

1) See "Maxim application note 85: Interfacing the DS1620 to the Motorola SPI Bus" for information on compatibility between 3-wire and SPI

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified



input/output, this gives a serious advantage in data transfer rates. The interface of many SPI-group components can currently be clocked at some tens of megahertz, further increasing the data rate advantage.

Unlike I²C, interface clock speed is not relying on a central standard, and can be set in a "least common denominator" fashion. Datarates of new SPI components can therefore be expected to rise continuously as the technology advances, without any changes to the standard being necessary.

3.2.2 The I²C group

The I²C groups main two advantages is the bus architecture and 2 wire communications (except CBus). The advantage of having a true bus architecture is less wiring needed since slave addresses are used instead of having separate wires to every slave device. It also allows for several masters to be connected to the same bus, something that would be harder to implement in a SPI-style system.

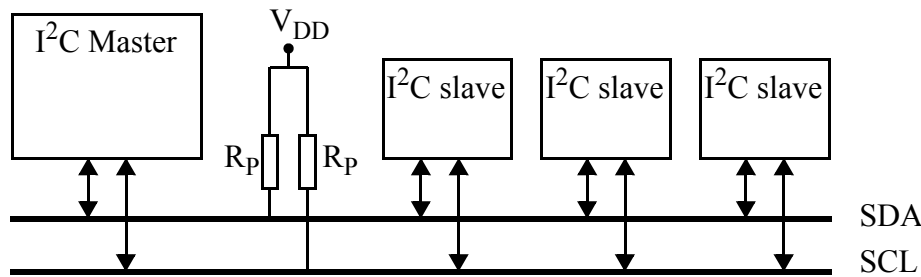


Figure 1: I²C bus

The masters tries to detect each other and backs of as soon as an other master seems to be accessing the bus. This normally means that the master unit that tries to use the bus first, will take the right to use it until it releases the bus into an idle state.

When the bus is idle, the SDA wire carries a logic '1'. Electrically this means that all units connected to the wire has set their output buffer in a high impedance state. The voltage level of the

	SDA - Serial Data	
	SCL - Serial Clock	
	R _P - Pull up resistor	
	V _{DD} - logic '1' source	

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

line is then pulled up, typically by a pull-up resistance (R_P) connected to V_{DD} . The logic '0' is accomplished by connecting the output buffer to ground. This means that any connected unit can force a logic '0' on to the bus, while a logic '1' is accomplished only when the outputs of all units are in high impedance mode.

There are currently three standardised speed modes in which I^2C components can operate, Standard (100kbit/s), Fast (400kbit/s) and Highspeed (3.4Mbit/s).

Standard and Fast mode are identical except for frequency and the possibility to run new devices in standard mode is mostly a backwards compatibility issue.

With the addition of highspeed mode, some changes in the protocol was made specifying among other things how to handle a bus when slave devices from different speed modes are connected. It also specifies the use of current sources in the master units to improve rise time for the SCL signal.

The SMBus protocol is based on standard mode I^2C and contains only minor differences, mainly to decrease power consumption as it was originally designed for use in SBS (Smart Battery System) in notebooks and hand held devices. It has no support for Fast or Highspeed modes.

Today it is often used on PC motherboards for temperature surveillance and similar tasks. The components on the market is therefore often thermometers, fan control chips and other types of circuits for system monitoring that does not need the higher bandwidth of fast/highspeed mode I^2C .

3.2.3 Current and future situation

A quick oversight of the serial components entering today's market reveals that the majority features some form of SPI-compatible interface. The ratio of SPI-family to I^2C -family components new on the market during the last year by five of the major semiconductor manufacturers²⁾ is roughly 10:1 and other serial interfaces are even less common.

Worth noticing is that even though SPI components outnumber I^2C , most manufacturers seem to attempt to offer a sufficiently large selection of I^2C components to satisfy the needs of most constructions. The reason for this is likely that the I^2C and SPI-families are used in two different ways and hence there is little reason to expect that any of them should disappear from the market any time shortly.

There are other interface standards that is used in specific areas, and will most likely live and prosper for years to come, such as LIN and CAN for the automotive industry, whereas others that try to compete in more general electronic construction does not seem to catch on.

2) Maxim, Analog devices, National semiconductors, Texas instruments and Philips

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

3.3 Choice of components and serial standard

3.3.1 Choice of communication standard for a design

Many factors come into play when deciding which communication standard to use for a design. The two groups of components (I²C and SPI families) specially studied in this thesis, have the major advantage of offering a wide range of components with built in support for the protocols.

Advantages of SPI

- Higher data transfer rates
- More components on the market
- Often supported directly from DSPs and similar components

Advantages of I²C

- Fewer wires even with only one slave device
- No additional wires per slave connected
- Better support for large number of slave devices
- Better reliability due to use of acknowledgment

Sometimes however, a design might have requirements that can not be fulfilled by either of these, and the attention then comes to the vast number of standards that can be supported by the use of separate driver components.

When using separate driver components, both power consumption, size and cost of the construction are most likely to rise. On the plus side, it gives a much wider choice of serial standards to use and components to attach.

One of the common requirements not fulfilled by I²C or SPI has to do with the range. If the intention is to communicate over long distances at high speeds, the use of optical or balanced (a.k.a. differential) signalling might be motivated.

There are a number of standards designed for this type of transfers and the most well known standard for this purpose is perhaps Ethernet, used in many modern computer networks. Modern versions of ethernet uses balanced signalling in pairs of twisted wires (TP-cables) to achieve minimum problems with interference.

A protocol that works in a similar fashion on the electrical level is LVDS (Low Voltage Differential Signalling). It may be a better candidate for more general designs as it is not as tightly bound to computer networking.

LVDS as specified in TIA/EIA-644-A is a standard developed for high transfer speeds. The specification mentioned specifies speeds up to 644mbps, although later implementations claims to have surpassed this. As its name indicates, it uses differential signalling to overcome much of the interference problems that long distances brings.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

The similarity between ethernet and LVDS in basic operation makes it possible for LVDS to use normal CAT5 cable which is normally used for ethernet, and due to its popularity, is relatively cheap compared to other cables of similar type.

Bused LVDS (BLVDS) is a modified specification made for connecting more units on the same cable.

An other option if reliability is a high priority is the 1553B bus which was created in the 1970's for military applications (originally for military aviation). It supports communication over relatively long distans (~300m) with multiple slave devices. It is a rather complex architecture suitable for large complex systems.

3.3.2 Choice of components for simulation in this thesis

Some components have been chosen for demonstration in hardware. The criterias for selecting components were that they should be from both the major families and that they should be in similar in function to enable a realistic evaluation of the differences in VHDL implementation of the masters. The components should also be general enough to be of interest to Saab Bofors dynamics as well as being easy to demonstrate without much extra equipment.

The components deemed easiest to demonstrate where thermometers since a correct function can easily be verified. I²C thermometers are quite common but SPI versions proved hard to find. The closest match were a 4 channel AD converter that contained a thermometer as an internal fifth channel.

EEPROM memories were also chosen as they are easy enough to demonstrate, and are often used for various applications.

Components from the I²C group:

- DS1621 - Maxim, Thermometer/thermostat.
- M24256-B - STMicroelectronics, Serial EEPROM 256kbit.

Components from SPI group:

- 25LC160 - Microchip, Serial EEPROM 16kbit.
- AD7817 - Analog Devices, 10bit 4 channel ADC with built in thermometer.

Most components in these groups is claimed to be "compatible" with one or several protocols. What this means in reality can vary between different manufacturers and components. Some components does not specify any compatibility at all but still implements some well known protocol to avoid licence costs. Close examination of the data sheet for a component is therefore unfortunately still necessary to be ensured that the component can be used with the planned master device.

25LC160 is described as having a SPI-compatible bus whereas AD7817 claims compatibility with SPI, QSPI and MICROWIRE. Both these components does however make use of wires that are unspecified in SPI but still affects the serial

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

transmission and therefore should be controlled by the master. The RD/ $\overline{\text{WR}}$ -pin on the AD7817 for instance is necessary for any kind of meaningful use of the device.

The data sheet of DS1621 describes a "Simple 2-wire serial interface" which in reality is the standard/fast mode I²C protocol. The name I²C is however never mentioned in this data sheet, probably due to license issues, and this seems to be a common method among manufacturers.

The M24256 EEPROM is specified as a "I²C bus EEPROM". The feature list in the data sheet mentions "Compatible with I²C extended addressing". This does not mean that the device uses 10bit addressing, but simply means that it can be connected to a I²C-bus where 10bit addressing is used with other devices without any problems. The compatibility with 10bit-devices is however a obligatory part of the I²C specification. This type of information seems to be added by manufacturers simply to make the most use of the licence when marketing the product and calls for caution when looking for components.

3.4 Functional description of lab implementation

3.4.1 Overview

The lab part of this thesis has been implemented on a FPGA development board from Memec. This is connected to a PC for user input/output via RS232.

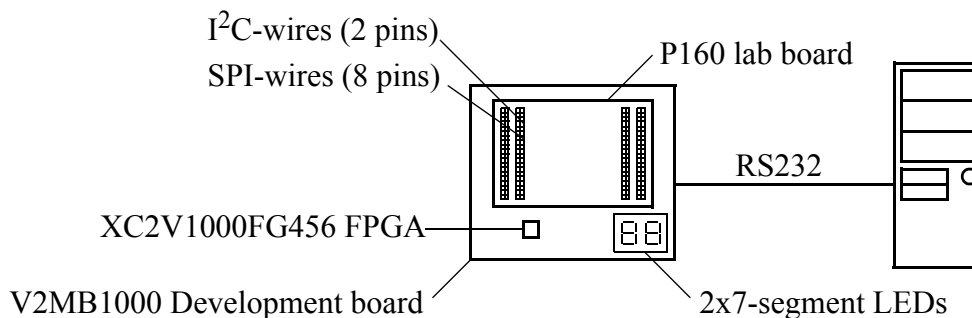


Figure 2: The development board

3.4.2 Development board

The FPGA on the development board is the Xilinx Virtex2. This is used to implement protocols for communicating with connected circuits (such as I²C slaves) as well as the computer used for input/output via RS232. It also features certain error detection which is indicated both via the 7-segment LED's and sent to the computer.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

3.4.3 P160 Lab board

The P160 lab board is used to physically connect external circuits to the FPGA. It is a circuit board that can be exchanged/removed from the rest of the evaluation board, and provides direct access to 160 pins on the FPGA, of which 109 are general IO-pins, the other 51 pins are used for GND, 2.5V or 3.3V feeds, JTAG, clock signals and so on.

Of the 109 IO pins, 2 are being used to represent the SDA and SCL lines of the I²C-bus in this design and another 8 pins are used for SPI.

3.4.4 PC

A software has been developed to communicate with the development board via the RS232 connection. This software has been developed on a PC running Gentoo Linux. The same computer has during the development phase been used to run the software for testing purposes of both the VHDL constructions on the development board and the software itself.

3.4.5 Software

Initially a simple text mode communication software was developed as an aid in the VHDL development process.

This communication software provides the user with a simple text interpreting environment and is started from a terminal with the number of the serial port connected to the development board as an argument.

Within the program a prompt is displayed and the program accepts input that either communicates with the FPGA or affects the program execution depending on input format. Two letter commands followed by four numbers of hexadecimal data can be input and sent to the development board.

```
SerCompMT>
Response: OK0001
SerCompMT>AD 0101
Input CMD: >>AD<< DATA: >>0101<<
SerCompMT>PT 1200
Input CMD: >>PT<< DATA: >>1200<<
SerCompMT>PT 0000
Input CMD: >>PT<< DATA: >>0000<<
Response: DO3FA0
SerCompMT>q
***Goodbye***
```

Figure 3: Reading temperature from AD7817

See **Table III:Commands** on page 20 for a complete list of available commands.

Data sent from the board is examined on every return strike and presented as text output in the same window as the input.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

The program also responds to some other commands. 'q' or 'quit' terminates the program and '?' displays a simple help message. A short message describing this is also shown at start of execution.

The second software was developed for demonstrative purposes and features a GUI to better visualize the possibilities of the project.

It allows users to for instance get a temperature reading from the AD7817 AD-converter by clicking on a button. In the background the program executes the same commands that are shown in **Figure 3: Reading temperature from AD7817** but the results are converted to centigrades before displaying them to the user.

It also allows the user to store and retrieve text from the EEPROMs in a similar way.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Table III: Commands

CMD	Data	Description
LT	<xxxx>	Loop This - this command will be returned as it was sent. If sent in lower case, upper case conversion will be applied.
LO	<0001>/<xxxx>	Loopback On - 0001 turns loopback on, all other values turns this off.
AD	<SA:AD>	ADdress - Last two characters indicates unit to be addressed, first two indicates sub-address if any.
PT	<DATA>	PuT data - Output DATA and address/sub address specified by AD command so that it can be handled by a connected driver. Sets the RnW to '0' to indicate a write operation.
GT	<DATA>	GeT data - Same as PT but sets the RnW line to '1'. No longer used by SPI and I ² C drivers.

3.4.6 Function of the VHDL constructions

An architectural overview of the VHDL blocks can be seen in **Figure 4: System overview** on page 23.

3.4.6.1 UART devices

An UART devices is used to communicate with a PC. The purpose is to provide a way of receiving commands and data to the on-board devices as well as sending results to be displayed to a user.

The connection to the computer is done with RS232 set to 115200kbps with 8bits of data per transfer and no parity bit.

The protocol for the communication is described in detail in section **4.4.1 Computer to Lab board RS232 interface**.

3.4.6.2 Protocol conversion

The unit for protocol conversion of incoming messages is responsible for checking that the message is valid and converting it to a interpretable format.

The STX and ETX bytes found first and last in each message are controlled and stripped by the "RecConv" unit and passed on to the "CmdIntrp" unit which converts the 12 bytes to 6 by interpreting them as ASCII as described in section **4.4.1 Computer to Lab board RS232 interface**.

Messages being output via the UART undergoes the same process in reverse.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

3.4.6.3 Command interpreter

This device interprets commands and decides what is to be done. Valid commands are forwarded to the device they regard and if invalid commands are encountered, a signal is sent to the error handler and the command is discarded.

See **Table III:Commands** on page 20 for a list of valid commands.

3.4.6.4 Device handler

To make the design as general as possible, the interface against the driver units is constructed in a bus -like fashion. Information is sent to the devices on two 16-bit wide buses, one for addresses and one for data, plus two 1-bit signals, one is the RnW (Read not write) signal and the other one the stroboscope ('1' when there are valid data on the bus).

Data is returned to the devicehandler through one 16-bit wide data bus per connected unit. A stroboscope signal is also added per unit, but only the device that are currently addressed are allowed to send data.

All units are expected to remember whether they were the one last addressed on the 16bit address bus. Data from units not currently addressed are ignored.

3.4.6.5 The I²C master

(For information on the I²C protocol, consult the I²C specification available from Philips Semiconductors.)

The I²C master is constructed to support a subset of the specification consisting of Standard/Fast mode transfers with 7 bit addressing. It also has the capability to use 10 bit addressing as described in chapter 4.3.1.3 on page 30

It receives data through and sends data via the interface described in chapter 4.4.2 on page 36 and uses internal FIFO units both for input and output.

It allows for internally (within the FPGA) connected I²C units and allows simulation models of I²C components to be tested and connected together with external physical components to ensure compatibility and to test the simulation model.

3.4.6.6 The SPI master

The SPI master controls the pins for SPI communication (MOSI, MISO, SCLK and SS) as well as some additional pins depending on the component that is to be communicated with (RD/ $\overline{RW}$ and $\overline{CONVST}$ for AD7817). The use of extra pins in SPI components prevents the master from being as generic as the I²C master, but the intention of the design is still to be as general as possible.

The SPI protocol specifies that each pin is used for one-way communication only, this means that all pins except MISO is outputs from the master.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

3.4.6.7 Error handler

The error handling unit takes input from a number of units and sends an error message through the RS232 connection via the output handler. To ensure the detection of an error, the two 7-segment user Led are also used to indicate error, in case there is a problem with the connection to the PC.

3.4.6.8 Loopback controller

All incoming commands are sent through the loopback controller which checks if the command is LT (Loop This) or LO (Loop On). All LT commands sent to the output controller. If the command "LO 0001" is detected, loopback is turned on and all commands are sent to output until another LO command is detected with a data argument other than "0001".

Loopback is turned off by default (after reset or reprogramming).

3.4.6.9 Output handler

There are three different components that can produce messages in command form (2 chars+4 hexadecimal numbers) to be output via RS232 unit. The output handler contains a separate FIFO for each one of these units to avoid information being lost due to collision. The state of the FIFOs are examined and data is sent as soon as one of them stops being empty. If more than one FIFO should contain data, the priority order is:

- 1) Error handler
- 2) Loop back
- 3) Device handler

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

4 System architectural design.

4.1 System overview

A basic overview of how the VHDL-blocks in the design are connected is seen in **Figure 4: System overview**. Basic data paths and external interfaces are indicated by arrows. The dotted arrows represents the signals for error indication,

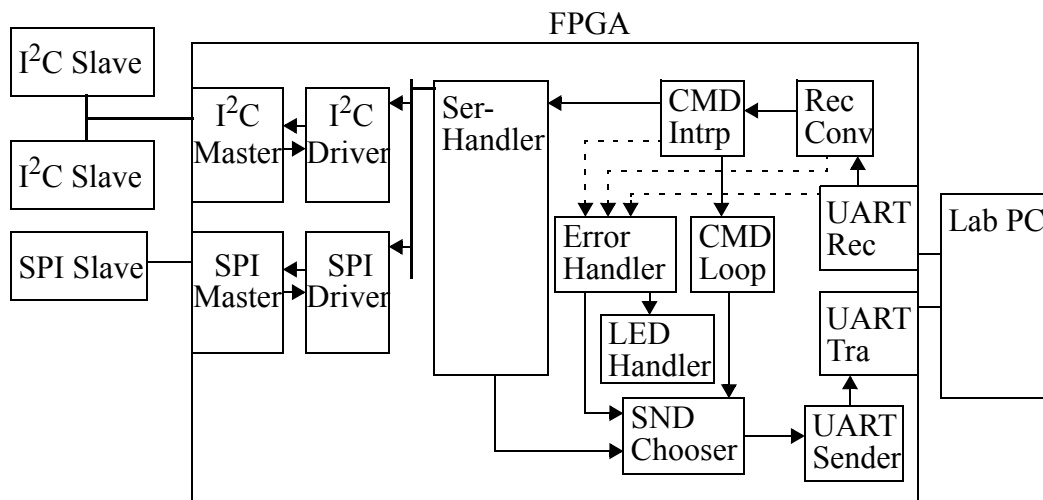


Figure 4: System overview

4.2 System components

4.2.1 Virtex2 lab board

The V2MB1000 lab board from Memec used in this project features a number of components of which the following are used in this thesis:

- XC2V1000FG456 FPGA
- JTAG (JTAG chain containing FPGA and PROM)
- PROM (Xilinx XC18V04) 512kb
- RS232 (TI MAX3221)
- P160 expansion module
- Two 7-segment LEDs
- Reset pushbutton

The design was loaded by using the JTAG interface to gain access to the PROM. The JTAG probe was connected to the parallel port on the PC. Loading the design from PROM into the FPGA is done either by the "program" push button, or by power cycling.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

4.2.2 PC platform for the software

The softwares for communicating with the evaluation board was written, compiled and run on a PC with Gentoo Linux, kernel 2.4.26-gentoo-r1, using gcc (Gnu C compiler) version 3.3.3.

The first software was written in C with the attempt of following the ANSI C standard.

The second software was developed in QT designer using Qt/C++ (Qt version 3.3.2).

A POSIX consistent environment was thereby provided which means that the software should be able to compile and run on most PCs with x86 linux and with minor modifications, on many other unix platforms.

4.2.3 The software

The first software was a simple text based program accepting command input on the form of "PT 1201". This was used during the development of the hardware constructions.

The source code for this software is partitioned in to three files.

- SerComp.c - source for the main program
- SerDriver.c - routines for RS232 handling
- SerRut.c - routines for protocol handling

SerDriver and SerRut also has header files, (SerDriver.h and SerRut.h).

The second software developed was attempted for demonstrative purposes. It used Qt/C++ which means that it can be compiled to work under a number of operating systems, including Linux and Windows.

It uses a library for communicating with the serial port that is not included in the Qt package (since the Qt package lacks such a library) called qextserialport. This project is open source and licensed under the GPL. It can be found on sourceforge (www.sourceforge.net/projects/qextserialport).

The purpose of this software is only to be used to utilize the communication between the computer and the connected components in order to show that the VHDL constructions in between works, and to demonstrate what the serial communication protocols (I²C and SPI) can be used for. It is therefore highly specialized and nearly useless for anything else.

It demonstrates that temperature readings can be retrieved from the thermometers and that text can be stored at different memory locations in the EEPROMS and then read back from those addresses.

4.2.4 I²C unit

The I²C device consists of three parts:

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

- Driver
- Master
- ANDer

4.2.4.1 I²C Driver Unit

The I²C driver unit accepts input from the SerHandler block via the interface described in **4.4.2 Interface with driver units** and stores it in a FIFO for the Master unit to retrieve.

It also reads a FIFO written by the Master unit and sends the information through the driver unit interface.

There are no direct interface between the Driver and Master units. All communication between the two are transferred through the two FIFOs in the Driver-part described in sections **4.2.4.2 Input FIFO** and **4.2.4.3 Output FIFO**.

This means that the input FIFO contains both data to be sent, information on data to retrieve and I²C addresses to communicate with.

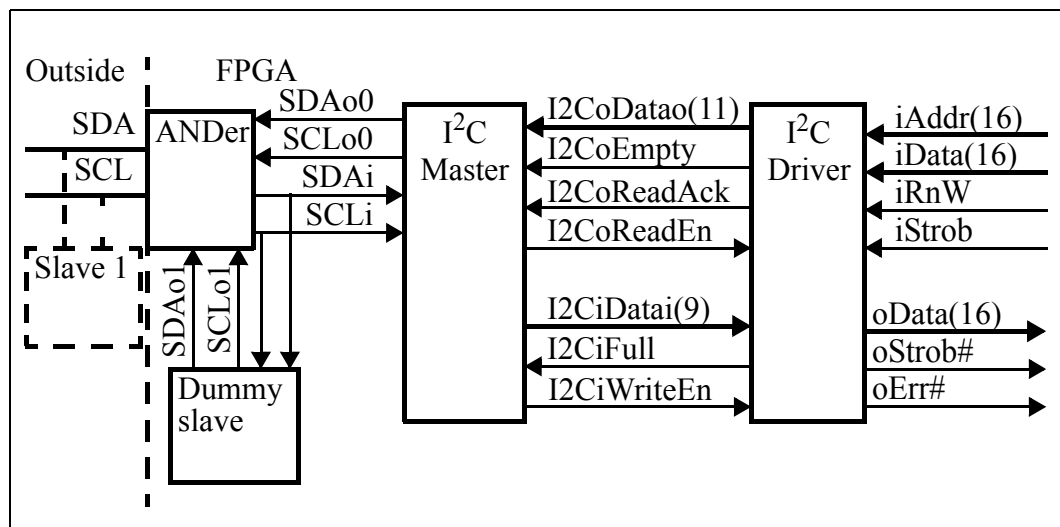


Figure 5: The I²C Master/Driver

4.2.4.2 Input FIFO

The input FIFO used to send information from the driver to the master unit is a 11bit wide 16bit deep synchronous FIFO generated from Coregen.

The empty flag of this FIFO is examined by the state machine of the I²C Master unit (see **4.3.1.4 State machine**) at certain states to determine what action is to be performed.

The three least significant bits of this FIFO contains the following information:

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

- Bit 0: If '1', a start (or restart) condition has to be sent before this byte is sent or read.
- Bit 1: If '1', a stop condition is to be sent after this byte has been sent or read.
- Bit 2: "Read-not-write" bit. If '1', read one byte, if '0', write the byte contained in Bit 3 to10.

The other eight bits contains either an I²C address to be sent, other data to be sent or, in the case of a read operation, just zeros that will be ignored.

4.2.4.3 Output FIFO

The output data is temporarily stored in a synchronous 9 bit wide 16 bit deep FIFO from coregen. In order to utilize the 16 bit wide driver interface, the bytes in this FIFO is sent in pairs by the Driver unit. Since an odd number of bytes may have to be sent, the least significant bit in the FIFO is interpreted as "Last byte in this batch" and if this is found to be '1', the data is sent even though only one byte has been read.

4.2.4.4 I²C Master unit

The I²C-Master unit can communicate with devices supporting the I²C protocol in standard or fast mode. Currently supported Features:

- 7 and 10 bit addressing.
- Standard or fast mode (configurable before synthesis).
- General call.

Unsupported features are:

- Master arbitration.
- Highspeed mode.
- Start byte

4.2.4.5 ANDer unit

The ANDer unit and the pinning description files provides the logic that connects the internal representation of the I²C bus with the external interfaces through two FPGA IO-pins used in "inout" mode. These two pins are connected to the P160 lab board which allows the connection of external devices to the bus. Since the I²C bus uses pull-up resistors to achieve a high level that any unit shall be able to lower, the output from the ANDer unit is 'Z' (high impedance output) or '0'.

All inter-FPGA I²C components connect to the ANDer unit by four connections: SCLi, SDAi, SCLo# and SDAo# where SDAi and SCLi are shared input signals

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

output from the ANDer unit. The signals output by the I²C units are numbered (SCLo0, SCLo1...) starting from 0 with the master.

4.2.5 SPI unit

The SPI unit consists of two parts in the same way as the I²C unit. A Driver unit to

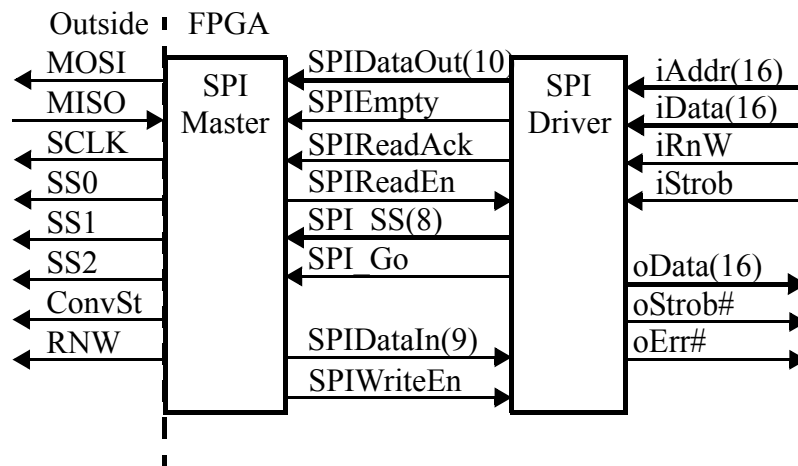


Figure 6: SPI Master/Driver

handle the communication with "SerHandler" driver interface, and a Master unit to perform the actual SPI operations.

4.2.5.1 The SPI Driver unit

In the same way as the I²C driver unit, the SPI driver unit contains two FIFOs. Both are synchronous and generated from Coregen. Unlike the I²C case, the SPI units communicates both via the FIFOs and by direct signalling.

The SPI_SS signal indicates witch slave device that shall be used. This signal is sent in parallel with the FIFO. The FIFO thereby only contains information on what is to be sent, as there are no real addresses in SPI unlike I²C.

The SPI_Go signal is also sent in parallel with the FIFO and indicates that information on an entire transaction currently exists in the FIFO. This is due to uncertainty about how some SPI slave devices handle long interrupts during transfers and may well be unnecessary.

The input FIFO is 10 bits wide and special information is stored in the two least significant bits:

- Bit 0 - Final Byte.
- Bit 1 - Read not write.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

The "Final Byte" bit is returned through the output FIFO as the least significant bit to indicate that the last byte is to be sent even if it is odd. Otherwise bytes are sent in pairs to fully utilize the 16 bit wide interface to the "SerHandler" unit.

4.2.5.2 **The SPI Master unit**

The SPI Master unit connects to the SPI slave devices by normal IN and OUT ports on the FPGA, routed via the P160 card. It consists of a small state machine (4 states) which reads information from the Driver part on what to do and carries out the tasks.

As well as the "normal" SPI wires (MOSI, MISO, SCLK and SS), this master also handles the CONVST (conversion start) and RNW (Read no write) wires required by the AD7817 AD-converter.

By default, the SPI-master operates the SPI interface at 0.3 MHz. This can be configured before synthesis if desired. An increase in this speed should not improve total system response time since the RS232 interface still creates a bottle neck at 115.2KHz with a large overhead.

The AD7817 is specified for 12.5MHz and the 25LC160 EEPROM for 2MHz.

4.2.6 **Error handler with LED interface**

The error handling device transmits error codes in two separate ways. The primary way is via the rs232 interface. This makes it possible to send a large variety of error codes and the order in which they are detected.

The secondary way is via the two 7-segment LED's on the board. This allows for certain error detection even if the RS232 interface for some reason does not work. When an error is reported, one of the LED's displays an 'E' to indicated that an error has occurred, while the other LED can be interpreted as an binary representation of the error code.

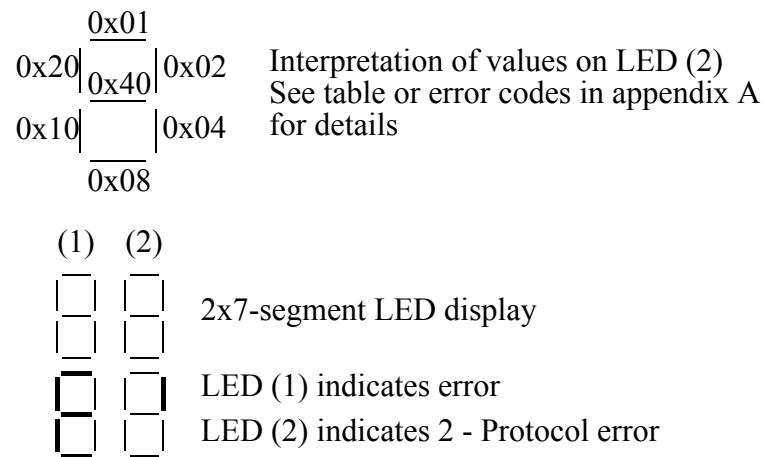


Figure 7: 7 segment user LEDs used for error indication

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

If more then one segment is lit on the second LED, then more than one error has occurred. It is not uncommon for several errors to be reported as a result of an incorrect command input. The LEDs will continue to indicate errors until the board is reset.

The system will attempt to carry on operation as usual even when errors are detected and if new errors are encountered at a later point, these will be added to the LED indication. See **Appendix A Error codes** for a complete list of error codes.

4.3 Concept of execution.

4.3.1 Operation of the I²C unit

4.3.1.1 General operation

All communication from the connected computer to the I²C master is performed using the "PT" command (and "AD" to set address).

PT-Command for I²C

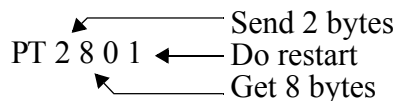


Figure 8: PT command example for I²C

The first PT-command is interpreted as a command according to **Figure 8: PT command example for I²C**. If the first number following "PT" is larger than zero, the master will expect more PT commands to supply data to be sent. In the previous example, 2 bytes are to be sent and these will be read from the following PT command.

EEPROM read example

AD5000	Set address
PT2801	Send 2 bytes, read 8
PT0480	2 bytes to send (1)=04="00000100" (2)=80="10000000"

Figure 9: I²C EEPROM read

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Figure 9: I2C EEPROM read shows a read operation from a M24256 EEPROM. To read from the EEPROM we first have to provide the internal memory address to the chip (2 bytes). The "PT0480" supplies the required 2 bytes of data and any PT-command following this will once again be interpreted as a command as in **Figure 8: PT command example for I2C**.

4.3.1.2 Addressing

Addresses are set using the "AD<SA:AD>" command (see **Table III: Commands**) where the "Address" part is used to indicate the I²C Unit. The address for the I²C unit is normally 00, although this may be set through a constant before compiling.

The "Sub address" part corresponds to the 7 bit I²C address of the slave that is to be used.

Available 7 bit slave addresses range from 000 1000 (0x08) to 111 0111 (0x77), see "The I²C-bus Specification" for a complete description of other addresses. This means that to address a slave device with address 0x08, the command "AD 0800" is sent, indicating sub address 08 (the I²C-slave) at address 00 (I²C-master).

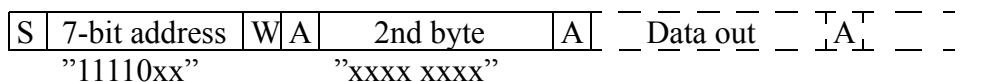
Observe that some I²C masters does not use the same way of describing the binary address as hex. The software for the "iPort AFM" I²C-master/slave device for RS232 for instance interprets 0001000 as 0x10 instead of 0x08. This makes sense if the RnW-bit sent after the 7 bit address is considered to be an 8th bit of the address.

For this reason, I²C addresses are normally provided in binary in data sheets and other documents.

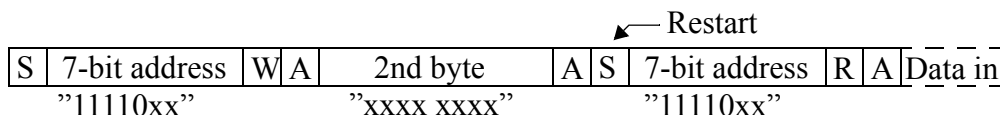
4.3.1.3 10 bit addressing

Since the 7 bits used for addressing only allows for a maximum of 128 addresses of which 16 are reserved for different purposes, an 10-bit extension of the addressing scheme is available. It is designed to be fully backwards compatible with 7 bit operation and uses one of the reserved 7 bit addresses to initiate transactions. **Figure 10: 10 bit addressing transactions** illustrates the use of 10 bit addresses. The reserved 7 bit address used is "11110xx" where the "xx" part is the first 2 bits of the 10 bit address. The other 8 bits are sent in the following byte.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified



1) Write mode



2) Read mode

xx xxxx xxxx - 10bit address
S - Start/Restart condition
W - Write
R - Read
A - Acknowledge

Figure 10: 10 bit addressing transactions

In case the master is performing a pure write operation ("Write mode" in **Figure 10: 10 bit addressing transactions**), this is rather trivial. The first byte are sent with the "Read-Not-Write" bit set to "Write" as usual, the following byte is the rest of the 10 bit address and the following bytes are the data that is to be sent.

In the other case, when the master reads from a slave with a 10 bit address, it gets a bit more complicated.

In order to maintain compatibility, the "Read-Not-Write" bit is still set to "Write" since the second part of the 10 bit address is still to be written. After this second byte, the transaction has to be restarted in order to change the direction of the data. It is then initialized with the same 7 bit address but this time followed by a "Read"-bit. The slave that recognized its full 10 bit address before restart remembers this and begins to send the requested data.

For a full description of 10 bit addressing, consult the I²C specifications (section 14 in version 2.1)

Since the master unit originally was built for 7-bit addressing only, 10-bit slaves have to be addressed using an extra byte supplied with the PT command.

Consider the following situation: A slave has the 10-bit address: "10 0001 0010" and 5 bytes is to be read from it. First a 7-bit address has to be sent, consisting of the 5 bits reserved for 10-bit addressing: "11110", and the first 2 bits of the 10 bit address: "10". This part can be handled with the normal "AD" command:

"11110"+"10"="111 1010"=7A and with the I²C-master at address "00" the addressing command has to be: "AD7A00".

After this, the rest of the address has to be sent, and this is done with "PT". The remaining part of the address is the byte: "0001 0010"=12. Since this single byte is

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

to be sent and 5 bytes is to be read, the PT-command has to be: "PT1501". The trailing '1' in the PT command indicates the use of the restart condition, and the 7 bit part of the address will always be sent after a restart condition (as described by the I²C specification).

4.3.1.4 State machine

The I²C master unit is realized as a state machine with seven different states. Directly after system reset, the state machine assumes the "Idle" state, at which the decision is made of what the next action will be.

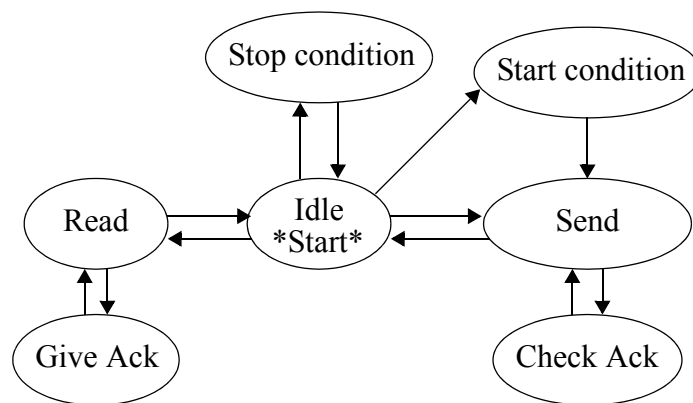


Figure 11: I²C Master unit state machine

This state is held until data appears in the input FIFO and the three least significant bits of the incoming data is then evaluated.

For information on the FIFO and how the data is interpreted, see **4.2.4.2 Input FIFO**.

The I²C protocol states that every transaction, including that of slave address, must be acknowledged. A slave may however temporarily choose not to respond to its address if it for instance is currently occupied handling some real time interrupt. This is handled with a time out counter which allows five retries before the address byte is discarded by the "Check Ack" state.

Since the current implementation of the master unit does not make any difference between addresses and other data coming from the input FIFO, this will also happen if a slave does not acknowledge a byte of data.

4.3.1.5 Future work

The parts if the I²C specification (version 2.1) not currently fulfilled by the master are:

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

- Highspeed mode
- Arbitration
- Start byte

Highspeed mode is perhaps both the hardest and the most attractive objective of these. It allows the master to communicate at speeds up to 3.4Mbit/s but requires extensive changes to the master. Changing the clock frequency as such is easy to do since this is related to constants. Even the 1:2 ratio between high and low period can be achieved easily since these uses separate constants.

All highspeed transactions must be started in either standard or fast mode, and handling the switch between high-speed and the other mode might prove somewhat harder to do. It will most likely required the addition of a number of states in the master state machine, as well as changes to the driver part and its input FIFO. The master has to be able to send a "Master address" to indicate the beginning of a highspeed-mode.

Currently the Master uses In and Out-ports to communicate with a "ANDer"-device with translates these to the InOut-mode used on the pins on the FPGA. The "ANDer" is responsible for conversion incoming '1' and '0' to outgoing 'Z' and '0' ('Z' being high impedance output). The electrical part of I²C high-speed specification specifies that the master providing the SCL signal shall use a current source to improve the rising time of the signal (that is, sending a '1' instead of a 'Z') with the exception of the time shortly after an 'Ack', when 'Z' has to be used to allow slaves to hold SCL low.

Timing the output of the three states of SCL ('1', '0' and 'Z') will require some extra work, and will be hard to realize using an "ANDer"-unit as it is used today.

Arbitration to allow more master devices on the bus can be achieved by altering the "Send" and "Start condition" states so that the bus is checked at all times. The specification states that this only has to be done until the address has been sent and acknowledged by a slave, but since the master currently does not make any difference between the sending of an address or data, the arbitration checking would be easiest to implement whenever something is sent or a start condition is to be performed.

The "Start byte" might be implemented by adding some states connected to the "Start cond". It will probably also require changes to the driver unit and the FIFO sending the data to the master unit.

Since the "Start byte" mostly used to allow connected microcontrollers to sample the I²C bus at a lower rate, it is considered a low priority in this implementation.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

4.3.2 Operation of the SPI unit

4.3.2.1 General operation

From the end users point of view, the SPI unit is operated in a similar way as the I²C unit. PT and AD commands are used much in the same way with the exception of the "restart" option in the I²C which does not have any equivalent in SPI.

PT-Command for SPI



Figure 12: PT command example for SPI

The first PT command is therefore used only to specify the number of bytes that are to be sent and received. The sending part always takes place before the reading which is performed directly after without any change to the SS lines (as sometimes required by SPI slaves).

4.3.2.2 Addressing

Addressing the SPI unit and its slaves are done with the AD command in the same way as the I²C unit. The main address of the SPI unit is 01 unless otherwise specified at compile time. There are no limit to the subaddresses due to the SPI protocol as such, but the theoretical limit due to the AD command would be 256 different slaves (AD0001 - ADFF01). The full 2 byte subaddress is forwarded to the SPI Master unit which could easily be modified to use more pins on the P160 board for SS (Slave Select) signals.

Currently only three pins are used for demonstration purposes, and only two of these are actually connected to a component.

4.3.2.3 Future work

Any future work on the SPI unit would be highly dependent on the specific future usage. Adding new SPI components may very well require changes in the inner workings.

What could be of a more general interest would be runtime configurable SCLK frequency allowing for different speeds against different units.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

4.4 Interface design

4.4.1 Computer to Lab board RS232 interface

The protocol for communication between the lab board and the PC states the following settings for RS232:

- 115200 bps.
- 8 data bits.
- no parity bit.

A normal D-sub 9 cable is used to physically connect the computer to the lab board. Only the TX and RX wire is used for the transmission.

On top of RS232, a protocol has been designed to transfer ASCII data. To avoid sending unforeseen control characters which could result in unforeseen behaviour, the message is encoded as 12 bytes of ASCII-HEX (0-9, A-F).

The message is started by a STX ("start of text", ASCII 02) character and ended with a ETX ("end of text", ASCII 03) which is added before and after the 12 characters, giving a total of 14 bytes.

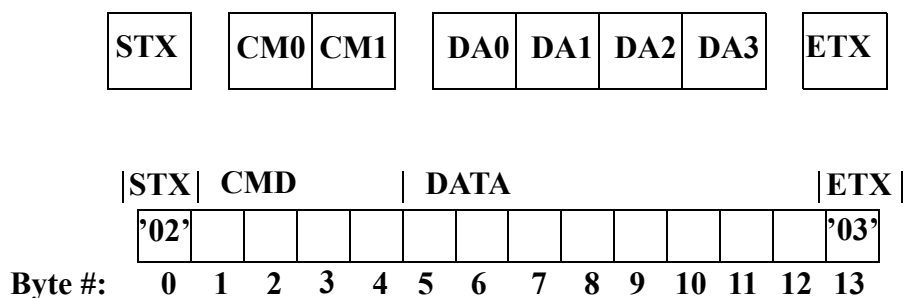


Figure 13: UART Protocol

If, for example, a "Loop this"-command is to be sent with the data part "1234" the original command would be: "LT1234". The ASCII representation of this would then be: "4C 45 31 32 33 34".

The STX and ETX parts are added here resulting in a message looking like:

" 'STX' 4C 45 31 32 33 34 'ETX' "

It is the ASCII representation of this message that is finally sent over the RS232 connection looking like:

"02 34 43 34 35 33 31 33 32 33 33 33 34 03".

If any control character would (by mistake) be entered in the original message, it would still be converted to ASCII.

A misplaced STX(="02" in ASCII) for instance would be sent as "30 32", the ASCII representation of the two numbers 0 and 2.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

4.4.2 Interface with driver units

All driver units connects to the following outputs from the "SerHandler" unit:

- iData
- iAddr
- iRnW
- iStrob

Where iData and iAddr are 16 bits wide. iRnW and iStrob are single wires. The iStrob wire goes high for one clock cycle when there are data on the bus, and all driver units are assumed to match the data of iAddr with its own address and receive the information on iData and iRnW on a match.

Another set of wires are connected for response:

- oData#
- oStrob#
- oErr#

The # states the number of the driver connected, this number will normally be used for driver address.

All information transmitted on these wires by a driver not currently addressed will be ignored. This is necessary since there is no other indication of which driver that sent the data.

4.5 Programmable logic requirements

Calculating the amount of programmable logic a design will consume of a programmable device is not an exact science. The result for a design may vary from time to time due to the elements of randomization in the place&route process.

The entire design claims roughly 33% of the space available on the VirtexII FPGA used in the project. Since the I²C and SPI parts might be reused in other designs, there was an interest in measuring the gates count for these parts separately.

By doing four separate compilations, as shown in **Table IV:Logic utilization** on page 37, the space requiered for these units was measured with a satisfacionary level of precision.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Table IV: Logic utilization

Configuration	Slice usage	Used 4 input LUTs	Equivalent gate count
I ² C+SPI	1719 (33%)	2247 (23%)	555,864
SPI	986 (19%)	1325 (12%)	345,829
I ² C	1476 (28%)	2155 (21%)	419,992
Without drivers	659 (12%)	1023 (9%)	208,272

By subtraction the size of the I²C and SPI drivers can be derived and the results are displayed in **Table V: SPI and I2C drivers size**. Keep in mind that these figures are not exact.

The “equivalent gate count” is made by Xilinx place&route tool to give a estimate on how many primitive gates the design would utilize and gives a fair estimate on design complexity.

Table V: SPI and I2C drivers size

Driver	Equivalent gate count
I ² C	210,000
SPI	136,000

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

5 Requirements traceability

The requirements of this thesis was to evaluate the market of serial digital components and give a brief overview of those that may be of interest for SAAB Bofors Dynamics and demonstrate the implementation of those standards and components that are of special interest.

The market evaluation revealed that two groups of standards, The I²C and SPI families, dominates the market. Other competing standards seems to have problems getting into the market.

This market evaluation and the result is presented in sections <3.1-3.3>.

Since the market proved to be dominated by two families of standards, the choice was made to implement these in VHDL and attempt to communicate with components that might be used in an actual product at SAAB.

The implementations of both standards were successfully implemented and tested against a number of components. Information on this is presented in sections <3.4> and chapter <4>.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

6 Notes

6.1 Explanation of terms:

FPGA - Fine Pitch Grid Array

A chip that is highly configurable and can be made to emulate nearly any digital components such as memories, processors etc.

The FPGA used in this thesis is the XC2V1000FG456 of the Virtex2 family from Xilinx. This device contains 11520 logic cells where one logic cell equals one 4-input LUT (Look up table) and one register. These can be configured and connected to create the desired component(s).

It also contains 720kbit of blockram which are configurable memory blocks, used in this thesis for a number of FIFO units.

The "FG456" part of the name indicates that the chip comes in a 456pin-package, and 324 of these are user configurable IO-pins. Of these 324 pins, 109 are connected to the P160 board (see **3.4.3 P160 Lab board**) and the others are connected to devices on the development board, such as the RS232 bus driver or the 7-segment LEDs.

EEPROM - Electrically Erasable Programmable Read Only Memory

Memory that when written keeps information even though power is disconnected. Used on the development board to keep the configuration of the FPGA so that a design can be kept on board when turned off. In this thesis, the EEPROM has been programmed through the JTAG chain.

JTAG / Boundary-scan - Joint Test Action Group

The JTAG/Boundary-scan technology was developed in the mid 1980's with the purpose of providing a standard for testing constructions. It is a 4 (optionally 5) wire protocol that specifies ways of testing components connected as way as providing a way of sending information to them.

On the development board used, the FPGA, EEPROM and P160 card are all connected to the boundary scan chain and this is used to program the EEPROM which then provides the programming for the FPGA.

Boundary scan = JTAG/IEEE 1149.1

Overhearing

A wire can pick up interference from surrounding wires and electronics due to the electromagnetic fields generated through altering currents.

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

6.2 Problems encountered and how to avoid them

6.2.1 Board reset

Please observe that the "reset" button on the memec development board is active low (= '0' when pressed). It also seems to give glitches although this has not yet been confirmed.

6.2.2 Downloading procedure

Downloading the design to the PROM and making it run can fail in so many ways due to problems with settings in the download software, Place&Route settings, jumper settings etc. Here is how it was done in the development of this thesis:

Mode select on board: Master Serial - All four jumpers are in place (closed).

Download cable: Insight JTAG cable (model IJC-2).

Operating system: Windows NT4.

Downloading software: iMPACT 4.1WP3.x

Table VI: iMPACT Settings

Mode	Boundary scan.
Erase before programming	Yes
Verify	No
Read protect	No
PROM Usercode	No
Skip user array	No
Load FPGA	No
Parallel mode	No

This impact version under this environment, fails to properly display the progress-bar. The downloading procedure is not affected by this, and the "Download successful" sign will appear after about 30 seconds as expected. The files used were created for loading into the EEPROM, and hence no file were ever downloaded directly to the FPGA.

P&R: The synthesis as well as the development was done from the tils_design_manager provided by Börje Karlsson (Krob, RTEDS) which runs Xilinx Place&Route configured from <some files. See appendix for these files?>

6.2.3 RS232

The TX and RX pins on the development board are switched in the manual, or should be seen from the view of the PC. Outputting data to the pin B7 or reading

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

data from A7 will fail. The signals are connected to a stand alone RS232 bus driver (TL...) which means that the problem can not be solved by simply switching RX and TX in the cable. It must be done right from the downloaded design.

The correct way is to use pin B7 as RX and A7 as TX in a following manner:

rx : in std_logic ;

tx : out std_logic ;

attribute pin_number of rx : signal is "B7 " ;

attribute pin_number of tx : signal is "A7 " ;

Date 2005-04-18	Edition 2	Document No TR-20050019
Document Status Released		Classification Unclassified

Appendix A Error codes

Table VII: Error code interpretation

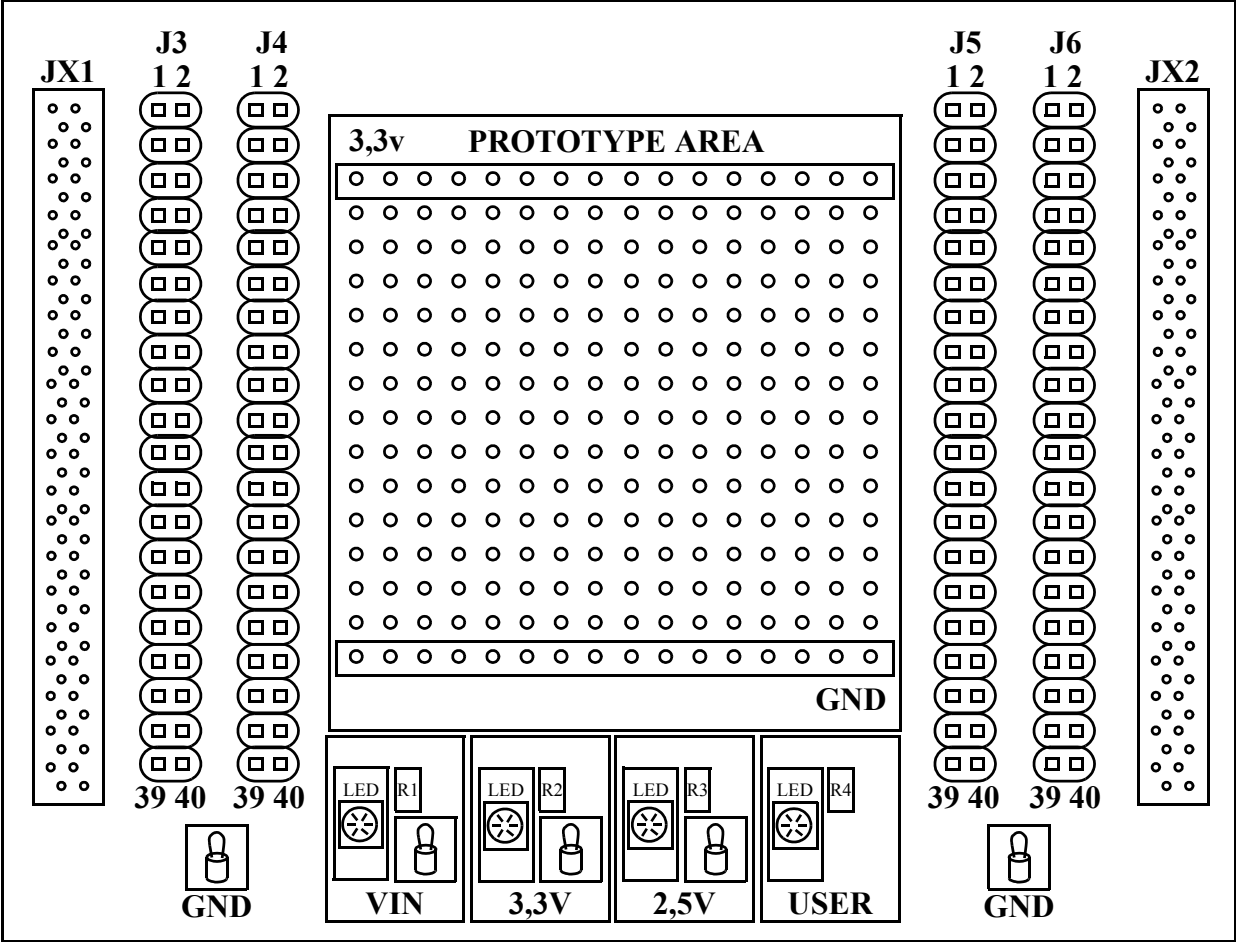
Error code	Meaning
0001	RX-error, wrong stop bit in RS232 transmission.
0002	Conversion Error, transmitted command contains other sign than 0-9, A-Z
0004	Protocol error, transmitted data does not start with "STX" or does not end with "ETX"
0008	Unknown command, not implemented
0010	Bad data format, data not in hex numbers (not yet implemented)
0020	Unexpected Data, does not match the given command (not yet implemented)
1<ab><c>	Device error, all error codes beginning with '1' originates from a "driver" device where "ab" is the number of the driver and "c" is the error (se below). (not yet implemented)
1xx0	Unknown error (not yet implemented)
1xx1	Addressed when busy, try again later (not yet implemented)
1xx2	No response from given address (in response to GT,PT,AD from I ² C-slave) (not yet implemented)

Date	2005-04-18	Edition	2	Document No	TR-20050019
Document Status	Released	Classification	Unclassified		

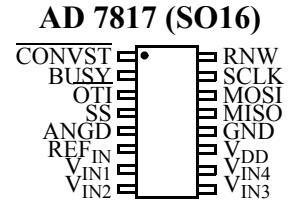
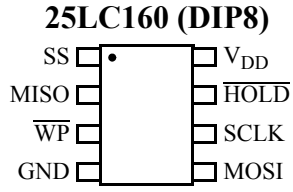
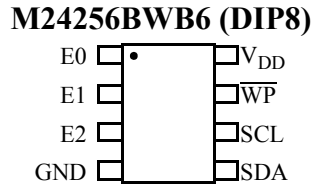
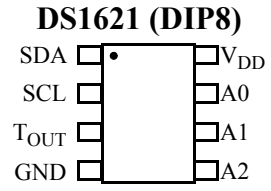
Table VIII: Connectors

Symbol	Explanation
JX1	P160 to main board connector (backside).
JX2	P160 to main board connector (backside).
J3	2x20 pin connector, (35 FP-GA-IO pins)
J4	2x20 pin connector, (19 FP-GA-IO pins)
J5	2x20 pin connector, (19 FP-GA-IO pins)
J6	2x20 pin connector, (36 FP-GA-IO pins)

Appendix B P160 board layout



Date	2005-04-18	Edition	2	Document No	TR-20050019
Document Status	Released	Classification	Unclassified		



Appendix C External circuits connection

